Sheet 11

Monday 22nd December, 2025

Please hand in a zip-archive, containing a pdf for exercise 1, as well as the plots and code for exercise 2, which is named <*firstname*>_<*lastname*>_*sheet*_<*number*>.

The deadline is Monday, Jan. 12th, 12:00 (noon). Please hand in your solution via moodle.

---

1 Least Squares Linear Regression

In the lecture we saw that one possible way to fit a function to data is using linear regression, with a least squares loss function. For every single data point y_i we check how far away the prediction $\boldsymbol{\omega}^\dagger \boldsymbol{x}_i$ is. The further apart data and prediction are, the worse our weights $\boldsymbol{\omega}$ are, so we want to minimize the least squares loss function to make prediction and data match as well as possible over all data-points. The loss function is defined as

$$E_{LS}(\boldsymbol{\omega}) = \frac{1}{2}\sum_{i=1}^{N}\left(\boldsymbol{\omega}^\dagger \boldsymbol{x}_i - y_i\right)^2. \tag{1}$$

A more elegant way to write this can be found by stacking the data points y_i into a vector $\boldsymbol{y}$ and the feature vectors $\boldsymbol{x}_i$ into a matrix X. We define

$$X_{ij} = (\boldsymbol{x}_i)_j, \tag{2}$$

resulting in the loss function

$$E_{LS}(\boldsymbol{\omega}) = \frac{1}{2}(X\boldsymbol{\omega} - \boldsymbol{y})^T(X\boldsymbol{\omega} - \boldsymbol{y}) \tag{3}$$

1. Show that the equations 1 and 3 are equivalent, by writing out the vector and matrix multiplications in eq. 3 in index form. *1 point*

2. Calculate the gradient of the loss function with respect to $\boldsymbol{\omega}$, explicitly showing that if A is symmetric
$$\nabla_\omega \boldsymbol{\omega}^T A\boldsymbol{\omega} = 2A\boldsymbol{\omega}. \tag{4}$$
2 points

3. Show that the loss function 3 is minimised by the weight-vector

$$\boldsymbol{\omega}^* = (X^T X)^{-1} X^T \boldsymbol{y}. \tag{5}$$

2 points

2 Toy Molecule

One major application for machine learning in materials science are machine learning force fields. The reason is quite simple: DFT-based molecular dynamics is accurate, but extremely expensive computationally. MD based on empirical force fields is cheap to use, but creating the force fields is difficult. By learning the energies and forces from DFT via machine learning, one easily acquires a classical force-field, with close to DFT accuracy.

In this exercise we will create a simple machine learning force field based on linear regression for a toy system: A single molecule consisting of 3 atoms called a, b and c. The energy is of course not a linear function of the atomic coordinates. However, to perform linear regression we don't need the function we're modelling to be linear in the feature vector, only linear in the weights $\boldsymbol{\omega}$.

Assuming the molecule is close to equilibrium, the total energy can be written as

$$E(\boldsymbol{x}_a, \boldsymbol{x}_b, \boldsymbol{x}_c) = \frac{1}{2}\left(k_{ab}(|\boldsymbol{x}_a - \boldsymbol{x}_b| - d_{ab})^2 + k_{ac}(|\boldsymbol{x}_a - \boldsymbol{x}_c| - d_{ac})^2 + k_{cb}(|\boldsymbol{x}_c - \boldsymbol{x}_b| - d_{cb})^2\right). \tag{6}$$

where k_{ij} are the force constants, and d_{ij} are the equilibrium distances between the atoms. We rewrite this expression for the energy to conform to the linear regression model, as

$$E(\boldsymbol{x}) = \sum_i \omega_i P_i(\boldsymbol{x}), \tag{7}$$

where $P_i(\boldsymbol{x})$ are all the polynomials of the pairwise distances of the atomic positions up to second order, e.g. $1, |\boldsymbol{x}_a - \boldsymbol{x}_b|, |\boldsymbol{x}_a - \boldsymbol{x}_b|^2, \ldots$. For each possible polynomial we have a coefficient ω_i, which is our weight vector. The force constants and equilibrium distances can easily be derived from the weight vector (Convince yourself that this is the case!).

Since we're only going to second order, we will not be able to capture anharmonic effects using this force field. However, we could easily extend it by simply adding further polynomial terms of the pairwise distances - we don't even have to stick to polynomials, and could use any functional dependence $\phi_i(\boldsymbol{x})$, leading to an energy of the form

$$E(\boldsymbol{x}) = \sum_i \omega_i \phi_i(\boldsymbol{x}). \tag{8}$$

This is usually referred to as using *basis functions* ϕ_i. Note that, from a physical perspective, the energy must be translation and rotation invariant; it is easy to show that

this is the case when using polynomials of the distances between the atoms, but not e.g. for using polynomials of the coordinates themselves. Including such considerations in the model leads to better matches with the real dynamics, even with a lower number of parameters.

1. Implement the function $E(\boldsymbol{\omega}, \boldsymbol{x})$ according to eq 7 which takes as arguments the weights $\boldsymbol{\omega}$ and positions of the three atoms $\boldsymbol{x}$, and outputs the energy.
   Hint: Your weight vector should have length 7.

   1 point

2. Implement a function which reads in the data from the *DATA.dat* file (Note that the starting velocities are zero, the collumns contain in order the time, the x,y,z components for the first atom, then those for the second and the third.). Calculate the matrix Φ, which contains the values for the basis functions of the structures $\boldsymbol{x}_j$, and the vector $\boldsymbol{E}$ which contains all energies from the training data. Make sure the basis functions for calculating Φ are ordered in the same way as they are for the weight vector ω. Return Φ and $\boldsymbol{E}$.

$$\Phi_{ij} = P_j(\boldsymbol{x}_i) \tag{9}$$

   1 point

3. Implement the least squares loss function $E_{LS}(\Phi, \boldsymbol{y}, \boldsymbol{\omega})$ according to eq 3. (Replace X with Φ for all definitions from exercise 1.)

   1 point

4. Now calculate the optimal weights using eq 5 (Replace X with Φ for all definitions from exercise 1.). What is the loss?

   1 point

5. Compute a MD-trajectory for the system using your calculated weights, and compare it with a plot of the original data. What do you observe, and what does it mean?

   1 point